```python
from pathlib import Path
import warnings
import json

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

try:
    import xgboost as xgb
except ImportError as e:
    raise ImportError("XGBoost is required. Install it with: pip install
xgboost") from e

warnings.filterwarnings("ignore")


# ==========================
# Configuration
# ==========================
TRAIN_FILE = "train.csv"
STORE_FILE = "store.csv"
OUTPUT_DIR = Path("outputs")

RANDOM_STATE = 42
SPLIT_RATIO = 0.90

MONTH_MAP = {
    1: "Jan", 2: "Feb", 3: "Mar", 4: "Apr", 5: "May", 6: "Jun",
    7: "Jul", 8: "Aug", 9: "Sep", 10: "Oct", 11: "Nov", 12: "Dec"
}
DOW_MAP = {
    0: "Mon", 1: "Tue", 2: "Wed", 3: "Thu", 4: "Fri", 5: "Sat", 6: "Sun"
}


# ==========================
# Utility functions
# ==========================
def ensure_dirs(base_dir: Path):
    charts_dir = base_dir / "charts"
    tables_dir = base_dir / "tables"
    charts_dir.mkdir(parents=True, exist_ok=True)
    tables_dir.mkdir(parents=True, exist_ok=True)
```

```python
    return charts_dir, tables_dir


def save_fig(fig, path: Path):
    fig.tight_layout()
    fig.savefig(path, dpi=300, bbox_inches="tight")
    plt.close(fig)


def rmspe(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    mask = y_true != 0
    if mask.sum() == 0:
        return np.nan
    return np.sqrt(np.mean(np.square((y_true[mask] - y_pred[mask]) /
y_true[mask])))


def rmse(y_true, y_pred):
    return float(np.sqrt(mean_squared_error(y_true, y_pred)))


# ========================
# Data loading and feature engineering
# ========================
def load_and_merge_data(train_path: str, store_path: str) -> pd.DataFrame:
    train = pd.read_csv(train_path, parse_dates=["Date"], low_memory=False)
    store = pd.read_csv(store_path, low_memory=False)
    df = pd.merge(train, store, on="Store", how="left")
    return df


def add_time_and_business_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    # Time features
    df["Year"] = df["Date"].dt.year
    df["Month"] = df["Date"].dt.month
    df["Day"] = df["Date"].dt.day
    df["DayOfWeekFromDate"] = df["Date"].dt.dayofweek
    df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)
    df["MonthStr"] = df["Month"].map(MONTH_MAP)
    df["DayName"] = df["DayOfWeekFromDate"].map(DOW_MAP)

    # Fill missing metadata
    if "CompetitionDistance" in df.columns:
        df["CompetitionDistance"] =
df["CompetitionDistance"].fillna(df["CompetitionDistance"].median())
```

```
    for col in ["CompetitionOpenSinceMonth", "CompetitionOpenSinceYear",
"Promo2SinceWeek", "Promo2SinceYear"]:
        if col in df.columns:
            df[col] = df[col].fillna(0)

    if "PromoInterval" in df.columns:
        df["PromoInterval"] = df["PromoInterval"].fillna("")
    else:
        df["PromoInterval"] = ""

    # Competition open duration
    df["CompetitionOpen"] = (
        12 * (df["Year"] - df["CompetitionOpenSinceYear"]) +
        (df["Month"] - df["CompetitionOpenSinceMonth"])
    )
    df["CompetitionOpen"] = df["CompetitionOpen"].clip(lower=0)

    # Promo open duration
    df["PromoOpen"] = (
        12 * (df["Year"] - df["Promo2SinceYear"]) +
        (df["WeekOfYear"] - df["Promo2SinceWeek"]) / 4.0
    )
    df["PromoOpen"] = df["PromoOpen"].clip(lower=0)

    # Promo month flag
    def check_promo_month(row):
        interval = row["PromoInterval"]
        if not isinstance(interval, str):
            return 0
        return int(row["MonthStr"] in interval)

    df["IsPromoMonth"] = df.apply(check_promo_month, axis=1)

    return df


def encode_for_model(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    if "StateHoliday" in df.columns:
        df["StateHoliday"] = df["StateHoliday"].astype(str).replace({
            "0": "0", "a": "1", "b": "2", "c": "3"
        }).astype(int)

    if "StoreType" in df.columns:
        df["StoreType"] = df["StoreType"].map({"a": 1, "b": 2, "c": 3, "d":
4}).fillna(0).astype(int)
```

```
    if "Assortment" in df.columns:
        df["Assortment"] = df["Assortment"].map({"a": 1, "b": 2, "c":
3}).fillna(0).astype(int)

    return df


def prepare_filtered_datasets(df_merged: pd.DataFrame):
    df_full = add_time_and_business_features(df_merged)

    eda_df = df_full.copy()

    model_df = df_full[(df_full["Open"] != 0) & (df_full["Sales"] > 0)].copy()
    model_df = encode_for_model(model_df)

    return eda_df, model_df


# ========================
# Train / test split and baselines
# ========================
def temporal_split(model_df: pd.DataFrame, split_ratio: float = 0.90):
    df = model_df.sort_values(["Date", "Store"]).reset_index(drop=True)
    split_index = int(len(df) * split_ratio)

    train_df = df.iloc[:split_index].copy()
    test_df = df.iloc[split_index:].copy()

    feature_drop = ["Sales", "Date", "Customers", "Open", "PromoInterval",
"MonthStr", "DayName", "DayOfWeek"]
    feature_drop = [c for c in feature_drop if c in df.columns]

    X_train = train_df.drop(columns=feature_drop)
    X_test = test_df.drop(columns=feature_drop)
    y_train = train_df["Sales"].copy()
    y_test = test_df["Sales"].copy()

    return train_df, test_df, X_train, X_test, y_train, y_test


def naive_store_dow_baseline(train_df: pd.DataFrame, test_df: pd.DataFrame) ->
np.ndarray:
    train = train_df[["Store", "DayOfWeekFromDate", "Sales"]].copy()
    test = test_df[["Store", "DayOfWeekFromDate"]].copy()

    g_store_dow = train.groupby(["Store", "DayOfWeekFromDate"],
as_index=False)["Sales"].mean()
    g_store = train.groupby("Store",
as_index=False)["Sales"].mean().rename(columns={"Sales": "Sales_store"})
```

```
    g_dow = train.groupby("DayOfWeekFromDate",
as_index=False)["Sales"].mean().rename(columns={"Sales": "Sales_dow"})
    global_mean = train["Sales"].mean()

    pred = test.merge(g_store_dow, on=["Store", "DayOfWeekFromDate"],
how="left")
    pred = pred.merge(g_store, on="Store", how="left")
    pred = pred.merge(g_dow, on="DayOfWeekFromDate", how="left")

    pred["Prediction"] = pred["Sales"]
    pred["Prediction"] = pred["Prediction"].fillna(pred["Sales_store"])
    pred["Prediction"] = pred["Prediction"].fillna(pred["Sales_dow"])
    pred["Prediction"] = pred["Prediction"].fillna(global_mean)

    return pred["Prediction"].to_numpy()


def fit_linear_regression(X_train: pd.DataFrame, y_train: pd.Series):
    model = LinearRegression()
    model.fit(X_train, y_train)
    return model


def fit_xgboost(X_train: pd.DataFrame, y_train: pd.Series, X_test:
pd.DataFrame, y_test: pd.Series):
    model = xgb.XGBRegressor(
        n_estimators=100,
        max_depth=8,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        objective="reg:squarederror"
    )
    model.fit(
        X_train,
        y_train,
        eval_set=[(X_train, y_train), (X_test, y_test)],
        verbose=False
    )
    return model


def build_metrics_table(y_true, pred_dict: dict) -> pd.DataFrame:
    rows = []
    for model_name, pred in pred_dict.items():
        rows.append({
            "Model": model_name,
```

```
            "RMSPE": rmspe(y_true, pred),
            "MAE": mean_absolute_error(y_true, pred),
            "RMSE": rmse(y_true, pred)
        })
    metrics = pd.DataFrame(rows).sort_values("RMSPE").reset_index(drop=True)
    return metrics


# =========================
# Plot helpers
# =========================
def plot_missing_values(df: pd.DataFrame, charts_dir: Path):
    missing = (df.isna().mean() * 100).sort_values(ascending=False)
    missing = missing[missing > 0]

    if missing.empty:
        return pd.DataFrame(columns=["Column", "MissingPct"])

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.bar(missing.index, missing.values)
    ax.set_title("Missing Values by Column (%)")
    ax.set_ylabel("Missing %")
    ax.set_xlabel("Column")
    ax.tick_params(axis="x", rotation=75)
    save_fig(fig, charts_dir / "01_missing_values.png")

    return pd.DataFrame({
        "Column": missing.index,
        "MissingPct": missing.values
    })


def plot_monthly_sales(eda_df: pd.DataFrame, charts_dir: Path):
    open_days = eda_df[(eda_df["Open"] != 0) & (eda_df["Sales"] > 0)].copy()
    monthly = open_days.groupby("Month", as_index=False)["Sales"].mean()
    monthly = monthly.set_index("Month").reindex(range(1, 13)).reset_index()
    monthly["MonthName"] = monthly["Month"].map(MONTH_MAP)

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.bar(monthly["MonthName"], monthly["Sales"])
    ax.set_title("Average Sales by Month")
    ax.set_ylabel("Average Sales")
    ax.set_xlabel("Month")
    ax.tick_params(axis="x", rotation=45)
    save_fig(fig, charts_dir / "02_monthly_avg_sales.png")

    return monthly
```

```python
def plot_weekday_sales(eda_df: pd.DataFrame, charts_dir: Path):
    open_days = eda_df[(eda_df["Open"] != 0) & (eda_df["Sales"] > 0)].copy()
    weekday = open_days.groupby("DayOfWeekFromDate",
as_index=False)["Sales"].mean()
    weekday =
weekday.set_index("DayOfWeekFromDate").reindex(range(7)).reset_index()
    weekday["DayName"] = weekday["DayOfWeekFromDate"].map(DOW_MAP)

    fig, ax = plt.subplots(figsize=(9, 5))
    ax.bar(weekday["DayName"], weekday["Sales"])
    ax.set_title("Average Sales by Day of Week")
    ax.set_ylabel("Average Sales")
    ax.set_xlabel("Day of Week")
    save_fig(fig, charts_dir / "03_weekday_avg_sales.png")

    return weekday


def plot_zero_sales_sundays(eda_df: pd.DataFrame, charts_dir: Path):
    temp = eda_df.copy()
    temp["IsSunday"] = (temp["DayOfWeekFromDate"] == 6).astype(int)
    summary = temp.groupby("IsSunday", as_index=False).agg(
        ZeroSalesShare=("Sales", lambda s: (s <= 0).mean()),
        ClosedShare=("Open", lambda s: (s == 0).mean())
    )
    summary["Group"] = summary["IsSunday"].map({0: "Non-Sunday", 1: "Sunday"})

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.bar(summary["Group"], summary["ZeroSalesShare"] * 100)
    ax.set_title("Share of Zero-Sales Days: Sunday vs Non-Sunday")
    ax.set_ylabel("Zero-Sales Share (%)")
    save_fig(fig, charts_dir / "04_sunday_zero_sales_share.png")

    return summary


def plot_promo_uplift(eda_df: pd.DataFrame, charts_dir: Path):
    open_days = eda_df[(eda_df["Open"] != 0) & (eda_df["Sales"] > 0)].copy()

    promo_stats = (
        open_days.groupby("Promo")["Sales"]
        .agg(MeanSales="mean", MedianSales="median", Count="count")
        .reset_index()
    )

    promo_stats["PromoLabel"] = promo_stats["Promo"].map({0: "No Promo", 1:
"Promo"})

    fig, ax = plt.subplots(figsize=(8, 5))
```

```
    ax.bar(promo_stats["PromoLabel"], promo_stats["MeanSales"])
    ax.set_title("Average Sales: Promo vs No Promo")
    ax.set_ylabel("Average Sales")
    save_fig(fig, charts_dir / "05_promo_avg_sales.png")

    no_promo_sample = open_days.loc[open_days["Promo"] == 0, "Sales"]
    promo_sample = open_days.loc[open_days["Promo"] == 1, "Sales"]

    if len(no_promo_sample) > 50000:
        no_promo_sample = no_promo_sample.sample(50000,
random_state=RANDOM_STATE)
    if len(promo_sample) > 50000:
        promo_sample = promo_sample.sample(50000, random_state=RANDOM_STATE)

    box_data = [no_promo_sample, promo_sample]

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.boxplot(box_data, labels=["No Promo", "Promo"], showfliers=False)
    ax.set_title("Sales Distribution: Promo vs No Promo")
    ax.set_ylabel("Sales")
    save_fig(fig, charts_dir / "06_promo_boxplot.png")

    return promo_stats


def plot_month_dow_heatmap(eda_df: pd.DataFrame, charts_dir: Path):
    open_days = eda_df[(eda_df["Open"] != 0) & (eda_df["Sales"] > 0)].copy()
    heat = open_days.pivot_table(
        index="Month",
        columns="DayOfWeekFromDate",
        values="Sales",
        aggfunc="mean"
    ).reindex(index=range(1, 13), columns=range(7))

    fig, ax = plt.subplots(figsize=(10, 6))
    im = ax.imshow(heat.values, aspect="auto")
    ax.set_title("Average Sales Heatmap: Month × Day of Week")
    ax.set_xlabel("Day of Week")
    ax.set_ylabel("Month")
    ax.set_xticks(range(7))
    ax.set_xticklabels([DOW_MAP[i] for i in range(7)])
    ax.set_yticks(range(12))
    ax.set_yticklabels([MONTH_MAP[i] for i in range(1, 13)])
    fig.colorbar(im, ax=ax)
    save_fig(fig, charts_dir / "07_month_dow_heatmap.png")

    heat_export = heat.copy()
    heat_export.index = [MONTH_MAP[i] for i in heat_export.index]
    heat_export.columns = [DOW_MAP[i] for i in heat_export.columns]
```

```python
    return heat_export


def plot_correlation_heatmap(model_df: pd.DataFrame, charts_dir: Path):
    cols = [
        "Sales", "Promo", "Promo2", "SchoolHoliday", "StateHoliday",
        "CompetitionDistance", "CompetitionOpen", "PromoOpen",
        "IsPromoMonth", "StoreType", "Assortment", "Store",
        "Month", "WeekOfYear", "DayOfWeekFromDate"
    ]
    cols = [c for c in cols if c in model_df.columns]

    corr = model_df[cols].corr(numeric_only=True)

    fig, ax = plt.subplots(figsize=(11, 9))
    im = ax.imshow(corr.values, aspect="auto", vmin=-1, vmax=1)
    ax.set_title("Correlation Heatmap")
    ax.set_xticks(range(len(corr.columns)))
    ax.set_xticklabels(corr.columns, rotation=90)
    ax.set_yticks(range(len(corr.index)))
    ax.set_yticklabels(corr.index)
    fig.colorbar(im, ax=ax)
    save_fig(fig, charts_dir / "08_correlation_heatmap.png")

    return corr


def plot_competition_distance_effect(model_df: pd.DataFrame, charts_dir:
Path):
    temp = model_df[["CompetitionDistance", "Sales"]].copy()
    temp = temp[temp["CompetitionDistance"].notna()].copy()

    temp["CompDistBin"] = pd.qcut(
        temp["CompetitionDistance"],
        q=10,
        duplicates="drop"
    )

    dist_summary = temp.groupby("CompDistBin", as_index=False)["Sales"].mean()
    dist_summary["BinLabel"] = dist_summary["CompDistBin"].astype(str)

    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot(range(len(dist_summary)), dist_summary["Sales"], marker="o")
    ax.set_title("Average Sales by Competition Distance Decile")
    ax.set_xlabel("Competition Distance Bin")
    ax.set_ylabel("Average Sales")
    ax.set_xticks(range(len(dist_summary)))
    ax.set_xticklabels(dist_summary["BinLabel"], rotation=60, ha="right")
    save_fig(fig, charts_dir / "09_competition_distance_bins.png")
```

```
    return dist_summary


def plot_storetype_promo_interaction(model_df: pd.DataFrame, charts_dir:
Path):
    temp = model_df.groupby(["StoreType", "Promo"],
as_index=False)["Sales"].mean()
    storetypes = sorted(temp["StoreType"].dropna().unique().tolist())

    width = 0.35
    x = np.arange(len(storetypes))
    no_promo = []
    yes_promo = []

    for st in storetypes:
        no_promo_val = temp[(temp["StoreType"] == st) & (temp["Promo"] ==
0)]["Sales"]
        yes_promo_val = temp[(temp["StoreType"] == st) & (temp["Promo"] ==
1)]["Sales"]
        no_promo.append(float(no_promo_val.iloc[0]) if len(no_promo_val) else
np.nan)
        yes_promo.append(float(yes_promo_val.iloc[0]) if len(yes_promo_val)
else np.nan)

    fig, ax = plt.subplots(figsize=(9, 5))
    ax.bar(x - width / 2, no_promo, width, label="No Promo")
    ax.bar(x + width / 2, yes_promo, width, label="Promo")
    ax.set_title("Average Sales by StoreType and Promo")
    ax.set_xlabel("StoreType")
    ax.set_ylabel("Average Sales")
    ax.set_xticks(x)
    ax.set_xticklabels([str(s) for s in storetypes])
    ax.legend()
    save_fig(fig, charts_dir / "10_storetype_promo_interaction.png")

    return temp


def plot_feature_importance(xgb_model, X_train: pd.DataFrame, charts_dir:
Path):
    fi = pd.DataFrame({
        "Feature": X_train.columns,
        "Importance": xgb_model.feature_importances_
    }).sort_values("Importance", ascending=False).reset_index(drop=True)

    top_fi = fi.head(15).iloc[::-1]

    fig, ax = plt.subplots(figsize=(9, 7))
```

```
    ax.barh(top_fi["Feature"], top_fi["Importance"])
    ax.set_title("Top 15 XGBoost Feature Importances")
    ax.set_xlabel("Importance")
    ax.set_ylabel("Feature")
    save_fig(fig, charts_dir / "11_feature_importance_top15.png")

    return fi


def build_prediction_frame(test_df: pd.DataFrame, y_test, pred_dict: dict):
    pred_df = test_df[["Date", "Store", "Promo", "StoreType", "Month",
"DayOfWeekFromDate"]].copy()
    pred_df["Actual"] = y_test.values

    for name, pred in pred_dict.items():
        pred_df[name] = pred

    pred_df["XGBoostResidual"] = pred_df["Actual"] - pred_df["XGBoost"]
    pred_df["XGBoostAbsError"] = np.abs(pred_df["XGBoostResidual"])
    pred_df["XGBoostAPE"] = pred_df["XGBoostAbsError"] / pred_df["Actual"]
    return pred_df


def plot_model_comparison(metrics_df: pd.DataFrame, charts_dir: Path):
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.bar(metrics_df["Model"], metrics_df["RMSPE"] * 100)
    ax.set_title("Model Comparison: RMSPE (%)")
    ax.set_ylabel("RMSPE (%)")
    ax.set_xlabel("Model")
    save_fig(fig, charts_dir / "12_model_comparison_rmspe.png")

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.bar(metrics_df["Model"], metrics_df["MAE"])
    ax.set_title("Model Comparison: MAE")
    ax.set_ylabel("MAE")
    ax.set_xlabel("Model")
    save_fig(fig, charts_dir / "13_model_comparison_mae.png")


def plot_actual_vs_pred_total(pred_df: pd.DataFrame, charts_dir: Path):
    daily = pred_df.groupby("Date", as_index=False)[["Actual",
"NaiveBaseline", "LinearRegression", "XGBoost"]].sum()

    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot(daily["Date"], daily["Actual"], label="Actual")
    ax.plot(daily["Date"], daily["NaiveBaseline"], label="Naive Baseline")
    ax.plot(daily["Date"], daily["LinearRegression"], label="Linear
Regression")
    ax.plot(daily["Date"], daily["XGBoost"], label="XGBoost")
```

```
    ax.set_title("Test Period: Actual vs Predicted Daily Sales")
    ax.set_xlabel("Date")
    ax.set_ylabel("Total Daily Sales")
    ax.legend()
    save_fig(fig, charts_dir / "14_actual_vs_pred_total_test.png")

    return daily


def plot_actual_vs_pred_sample_stores(pred_df: pd.DataFrame, charts_dir: Path,
top_n=3):
    top_stores = (
        pred_df.groupby("Store", as_index=False)["Actual"]
        .sum()
        .sort_values("Actual", ascending=False)
        .head(top_n)["Store"]
        .tolist()
    )

    exported = []
    for store_id in top_stores:
        temp = pred_df[pred_df["Store"] ==
store_id].sort_values("Date").copy()

        fig, ax = plt.subplots(figsize=(12, 5))
        ax.plot(temp["Date"], temp["Actual"], label="Actual")
        ax.plot(temp["Date"], temp["XGBoost"], label="XGBoost")
        ax.set_title(f"Store {store_id}: Actual vs XGBoost Prediction")
        ax.set_xlabel("Date")
        ax.set_ylabel("Sales")
        ax.legend()
        file_name = f"15_store_{store_id}_actual_vs_pred.png"
        save_fig(fig, charts_dir / file_name)
        exported.append({"Store": store_id, "File": file_name})

    return pd.DataFrame(exported)


def plot_residuals(pred_df: pd.DataFrame, charts_dir: Path):
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.hist(pred_df["XGBoostResidual"], bins=50)
    ax.set_title("XGBoost Residual Distribution")
    ax.set_xlabel("Residual")
    ax.set_ylabel("Frequency")
    save_fig(fig, charts_dir / "16_residual_histogram.png")

    daily_resid = pred_df.groupby("Date",
as_index=False)["XGBoostResidual"].mean()
    fig, ax = plt.subplots(figsize=(12, 5))
```

```python
    ax.plot(daily_resid["Date"], daily_resid["XGBoostResidual"])
    ax.axhline(0, linestyle="--")
    ax.set_title("Average XGBoost Residual Over Time")
    ax.set_xlabel("Date")
    ax.set_ylabel("Average Residual")
    save_fig(fig, charts_dir / "17_residuals_over_time.png")

    return daily_resid


def plot_error_heatmaps(pred_df: pd.DataFrame, charts_dir: Path):
    temp = pred_df.copy()
    temp["MonthName"] = temp["Month"].map(MONTH_MAP)
    temp["DayName"] = temp["DayOfWeekFromDate"].map(DOW_MAP)

    heat1 = temp.pivot_table(
        index="Month",
        columns="Promo",
        values="XGBoostAPE",
        aggfunc="mean"
    ).reindex(index=range(1, 13))

    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(heat1.values, aspect="auto")
    ax.set_title("Mean Absolute % Error: Month × Promo")
    ax.set_xlabel("Promo")
    ax.set_ylabel("Month")
    ax.set_xticks(range(len(heat1.columns)))
    ax.set_xticklabels([f"Promo={c}" for c in heat1.columns])
    ax.set_yticks(range(12))
    ax.set_yticklabels([MONTH_MAP[i] for i in range(1, 13)])
    fig.colorbar(im, ax=ax)
    save_fig(fig, charts_dir / "18_error_heatmap_month_promo.png")

    heat2 = temp.pivot_table(
        index="Month",
        columns="DayOfWeekFromDate",
        values="XGBoostAPE",
        aggfunc="mean"
    ).reindex(index=range(1, 13), columns=range(7))

    fig, ax = plt.subplots(figsize=(10, 6))
    im = ax.imshow(heat2.values, aspect="auto")
    ax.set_title("Mean Absolute % Error: Month × Day of Week")
    ax.set_xlabel("Day of Week")
    ax.set_ylabel("Month")
    ax.set_xticks(range(7))
    ax.set_xticklabels([DOW_MAP[i] for i in range(7)])
    ax.set_yticks(range(12))
```

```
    ax.set_yticklabels([MONTH_MAP[i] for i in range(1, 13)])
    fig.colorbar(im, ax=ax)
    save_fig(fig, charts_dir / "19_error_heatmap_month_dow.png")

    return heat1, heat2


def plot_store_level_error(pred_df: pd.DataFrame, charts_dir: Path):
    store_err = pred_df.groupby("Store", as_index=False).agg(
        ActualSales=("Actual", "sum"),
        MAE=("XGBoostAbsError", "mean"),
        RMSPE=("XGBoostAPE", lambda s: np.sqrt(np.mean(np.square(s))))
    ).sort_values("MAE", ascending=False)

    top20 = store_err.head(20).sort_values("MAE", ascending=True)

    fig, ax = plt.subplots(figsize=(10, 8))
    ax.barh(top20["Store"].astype(str), top20["MAE"])
    ax.set_title("Top 20 Stores by XGBoost MAE")
    ax.set_xlabel("MAE")
    ax.set_ylabel("Store")
    save_fig(fig, charts_dir / "20_top20_store_mae.png")

    return store_err


# =========================
# Summary writer
# =========================
def write_summary_markdown(
    base_dir: Path,
    df_merged: pd.DataFrame,
    eda_df: pd.DataFrame,
    model_df: pd.DataFrame,
    metrics_df: pd.DataFrame,
    monthly_df: pd.DataFrame,
    promo_stats: pd.DataFrame,
    fi_df: pd.DataFrame,
    daily_pred_df: pd.DataFrame,
    store_err_df: pd.DataFrame
):
    start_date = df_merged["Date"].min()
    end_date = df_merged["Date"].max()
    n_rows = len(df_merged)
    n_stores = df_merged["Store"].nunique()

    best_model = metrics_df.iloc[0]
    december_sales = monthly_df.loc[monthly_df["Month"] == 12,
"Sales"].iloc[0]
```

```
    best_month_row = monthly_df.sort_values("Sales", ascending=False).iloc[0]

    no_promo_mean = promo_stats.loc[promo_stats["Promo"] == 0,
"MeanSales"].iloc[0]
    promo_mean = promo_stats.loc[promo_stats["Promo"] == 1,
"MeanSales"].iloc[0]
    promo_uplift = (promo_mean / no_promo_mean - 1) if no_promo_mean > 0 else
np.nan

    top_features = fi_df.head(5)["Feature"].tolist()
    worst_store = store_err_df.iloc[0]

    summary_lines = [
        "# Rossmann PPT Pack Summary",
        "",
        "## Dataset overview",
        f"- Merged rows: {n_rows:,}",
        f"- Number of stores: {n_stores:,}",
        f"- Date range: {start_date.date()} to {end_date.date()}",
        f"- Rows used for model after filtering closed/zero-sales days:
{len(model_df):,}",
        "",
        "## EDA highlights",
        f"- Highest average sales month: {best_month_row['MonthName']}
({best_month_row['Sales']:,.0f})",
        f"- December average sales: {december_sales:,.0f}",
        f"- Promo uplift in mean sales: {promo_uplift * 100:.2f}%",
        "",
        "## Model performance",
        f"- Best RMSPE model: {best_model['Model']} ({best_model['RMSPE'] *
100:.2f}%)",
        f"- Best MAE model: {metrics_df.sort_values('MAE').iloc[0]['Model']}
({metrics_df.sort_values('MAE').iloc[0]['MAE']:,.0f})",
        "",
        "## Top XGBoost features",
        *[f"- {i + 1}. {feat}" for i, feat in enumerate(top_features)],
        "",
        "## Diagnostics",
        f"- Worst store by MAE: Store {int(worst_store['Store'])} with MAE
{worst_store['MAE']:,.0f}",
        "",
        "## Recommended slide order",
        "- 1. Business context + dataset overview",
        "- 2. Missing values / data quality",
        "- 3. Monthly seasonality",
        "- 4. Weekday pattern + Sunday zero-sales chart",
        "- 5. Promo impact chart",
        "- 6. Month × DayOfWeek heatmap",
        "- 7. Correlation heatmap",
```

```
        "- 8. Competition-distance effect",
        "- 9. Model comparison (Naive vs Linear vs XGBoost)",
        "- 10. Actual vs predicted on the test period",
        "- 11. Feature importance",
        "- 12. Residuals and error heatmaps",
        "- 13. Business implications"
    ]

    (base_dir / "summary.md").write_text("\n".join(summary_lines),
encoding="utf-8")


# ========================
# Main pipeline
# ========================
def main():
    charts_dir, tables_dir = ensure_dirs(OUTPUT_DIR)

    print("1) Loading and merging data...")
    df_merged = load_and_merge_data(TRAIN_FILE, STORE_FILE)

    print("2) Preparing EDA and model datasets...")
    eda_df, model_df = prepare_filtered_datasets(df_merged)

    print("3) Splitting data temporally...")
    train_df, test_df, X_train, X_test, y_train, y_test =
temporal_split(model_df, SPLIT_RATIO)

    print("4) Training baselines and XGBoost...")
    naive_pred = naive_store_dow_baseline(train_df, test_df)

    lin_model = fit_linear_regression(X_train, y_train)
    lin_pred = lin_model.predict(X_test)
    lin_pred = np.clip(lin_pred, a_min=0, a_max=None)

    xgb_model = fit_xgboost(X_train, y_train, X_test, y_test)
    xgb_pred = xgb_model.predict(X_test)
    xgb_pred = np.clip(xgb_pred, a_min=0, a_max=None)

    pred_dict = {
        "NaiveBaseline": naive_pred,
        "LinearRegression": lin_pred,
        "XGBoost": xgb_pred
    }

    print("5) Building metrics...")
    metrics_df = build_metrics_table(y_test, pred_dict)
    metrics_df.to_csv(tables_dir / "model_metrics.csv", index=False)
```

```
    print("6) Exporting charts and tables...")

    missing_df = plot_missing_values(df_merged, charts_dir)
    missing_df.to_csv(tables_dir / "missing_values.csv", index=False)

    monthly_df = plot_monthly_sales(eda_df, charts_dir)
    monthly_df.to_csv(tables_dir / "monthly_avg_sales.csv", index=False)

    weekday_df = plot_weekday_sales(eda_df, charts_dir)
    weekday_df.to_csv(tables_dir / "weekday_avg_sales.csv", index=False)

    sunday_df = plot_zero_sales_sundays(eda_df, charts_dir)
    sunday_df.to_csv(tables_dir / "sunday_zero_sales_summary.csv",
index=False)

    promo_stats = plot_promo_uplift(eda_df, charts_dir)
    promo_stats.to_csv(tables_dir / "promo_sales_summary.csv", index=False)

    heat_month_dow = plot_month_dow_heatmap(eda_df, charts_dir)
    heat_month_dow.to_csv(tables_dir / "month_dow_heatmap_values.csv")

    corr_df = plot_correlation_heatmap(model_df, charts_dir)
    corr_df.to_csv(tables_dir / "correlation_matrix.csv")

    comp_summary = plot_competition_distance_effect(model_df, charts_dir)
    comp_summary.to_csv(tables_dir / "competition_distance_summary.csv",
index=False)

    interaction_df = plot_storetype_promo_interaction(model_df, charts_dir)
    interaction_df.to_csv(tables_dir / "storetype_promo_interaction.csv",
index=False)

    fi_df = plot_feature_importance(xgb_model, X_train, charts_dir)
    fi_df.to_csv(tables_dir / "xgboost_feature_importance.csv", index=False)

    pred_df = build_prediction_frame(test_df, y_test, pred_dict)
    pred_df.to_csv(tables_dir / "test_predictions_detailed.csv", index=False)

    plot_model_comparison(metrics_df, charts_dir)

    daily_pred_df = plot_actual_vs_pred_total(pred_df, charts_dir)
    daily_pred_df.to_csv(tables_dir / "daily_test_predictions.csv",
index=False)

    sample_store_files = plot_actual_vs_pred_sample_stores(pred_df,
charts_dir, top_n=3)
    sample_store_files.to_csv(tables_dir / "sample_store_chart_files.csv",
index=False)
```

```python
    daily_resid_df = plot_residuals(pred_df, charts_dir)
    daily_resid_df.to_csv(tables_dir / "daily_residuals.csv", index=False)

    heat1, heat2 = plot_error_heatmaps(pred_df, charts_dir)
    heat1.to_csv(tables_dir / "error_heatmap_month_promo.csv")
    heat2.to_csv(tables_dir / "error_heatmap_month_dow.csv")

    store_err_df = plot_store_level_error(pred_df, charts_dir)
    store_err_df.to_csv(tables_dir / "store_level_errors.csv", index=False)

    dataset_summary = pd.DataFrame([
        {"Metric": "Merged rows", "Value": len(df_merged)},
        {"Metric": "Stores", "Value": df_merged["Store"].nunique()},
        {"Metric": "Date min", "Value": str(df_merged["Date"].min().date())},
        {"Metric": "Date max", "Value": str(df_merged["Date"].max().date())},
        {"Metric": "Rows used in model", "Value": len(model_df)},
        {"Metric": "Train rows", "Value": len(train_df)},
        {"Metric": "Test rows", "Value": len(test_df)},
    ])
    dataset_summary.to_csv(tables_dir / "dataset_summary.csv", index=False)

    write_summary_markdown(
        OUTPUT_DIR,
        df_merged,
        eda_df,
        model_df,
        metrics_df,
        monthly_df,
        promo_stats,
        fi_df,
        daily_pred_df,
        store_err_df
    )

    best_rmspe_row = metrics_df.sort_values("RMSPE").iloc[0]
    best_mae_row = metrics_df.sort_values("MAE").iloc[0]
    kpis = {
        "best_rmspe_model": best_rmspe_row["Model"],
        "best_rmspe_value": float(best_rmspe_row["RMSPE"]),
        "best_mae_model": best_mae_row["Model"],
        "best_mae_value": float(best_mae_row["MAE"]),
        "top_5_xgb_features": fi_df.head(5)["Feature"].tolist()
    }
    (OUTPUT_DIR / "kpis.json").write_text(json.dumps(kpis, indent=2),
encoding="utf-8")

    print("\nDone.")
    print(f"Outputs saved to: {OUTPUT_DIR.resolve()}")
    print("Main files:")
```

```python
    print("- outputs/charts/*.png")
    print("- outputs/tables/*.csv")
    print("- outputs/summary.md")
    print("- outputs/kpis.json")


if __name__ == "__main__":
    main()
```